

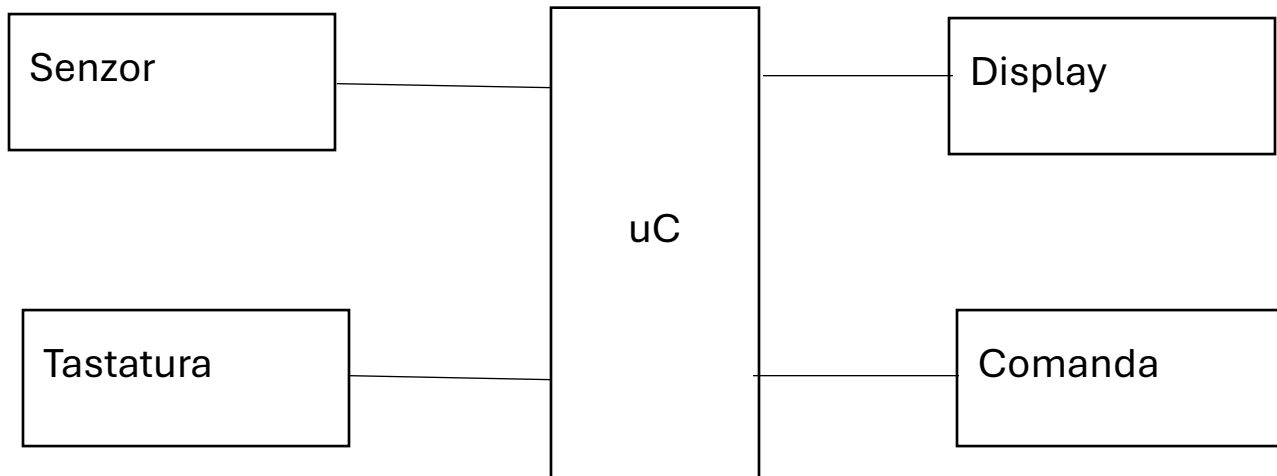
Proiect uC-Termostat de camera

Rîmbulea Robert,gr2133,sg3

Cuprins

1.Schema bloc.....	3
2.Categorii de senzori.....	4
3.Senzorul LM35.....	7
4.Dimensionarea rezistentelor.....	9
5.Convertorul ADC0808.....	11
6.Tabel pentru alegerea microcontrolerului.....	13
7.Adaugarea LCD-ului in proteus.....	14
8.Conexiuni.....	18
9.Organigrama.....	19
10.Codul in C.....	19
11.Simularea la 5 grade.....	22
12.Simularea la 30 de grade.....	23
13.Adaugarea releului in proteus.....	24
14.Adaugarea butoanelor in proteus.....	25
15.Circuitul complet.....	26
16.Codul final in C.....	27
17.Verificarea codului in proteus.....	31

1)Schema bloc:



Este prezentata schema bloc pentru un sistem de termostat de camera bazat pe un microcontroler.

Sistemul este compus din următoarele blocuri principale:

- Senzor:** Masoara temperatura si trimite date catre microcontroler.
- Microcontroler :** Proceaseaza datele primite si controleaza afisajul si comenzile.
- Display:** Prezinta temperatura curenta.
- Comanda:** Permite utilizatorului sa seteze temperatura dorita.

Specificatii:

-Temperatura: 5°C-30°C

-Precizia: 0.1°C-0.2°C

2) Categoriile de senzori:

2.1) Senzori de temperatura PTC

- **Senzorii PTC** sunt realizați din materiale care au un coeficient de temperatură pozitiv. La temperaturi scăzute, rezistența senzorului PTC este mică și curentul electric poate circula ușor prin el. Senzorii PTC sunt folosiți în termostate pentru a monitoriza temperatura și pentru a ajuta la controlul unui circuit. Pot fi utilizați în aplicații de măsurare a temperaturii, precum termostatele de cameră. Astfel, senzorii PTC sunt ideali pentru situații unde este necesară monitorizarea și controlul temperaturii, având în același timp și rol de protecție în cazul temperaturilor periculoase de ridicate.

-Avantaje: Unul dintre cele mai mari avantaje ale senzorilor PTC este capacitatea lor de auto-limitare a curentului la temperaturi ridicate.

2.2) Senzori de temperatura NTC

-Un **senzor NTC (Negative Temperature Coefficient)** este un **termistor** (rezistor dependent de temperatură) care își **scade rezistența pe măsură ce temperatura crește**. Acest comportament este opus față de senzorii PTC, care își cresc rezistența odată cu temperatura. Senzorii NTC sunt foarte sensibili la schimbările de temperatură, ceea ce îi face potriviți pentru măsurători precise.

-Avantaje: Se folosesc frecvent în termostatele de cameră pentru controlul precis al temperaturii. Fiind mici și sensibili, acești senzori detectează foarte repede schimbările de temperatură. Sunt mai ieftinoși o precizie excelentă pentru costul lor. Un NTC poate fi folosit direct cu un divizor de tensiune simplu.

2.3) Senzorii de temperatură cu semiconductori

-Acești senzori sunt fabricați folosind diodă, tranzistori sau circuite integrate pentru a măsura temperatura. Principiul lor de funcționare se bazează pe modificarea tensiunii sau curentului unui element semiconductor în funcție de temperatură.

-Avantaje: Au o liniaritate excelentă, ceea ce îi face ușor de utilizat fără calibrare complex, pot fi conectați direct la microcontrolere fără circuite suplimentare, si sunt ieftini și au o durată de viață lungă datorită absenței pieselor mobile.

2.4) Senzorii de temperatură rezistivi (RTD)

Creșterea temperaturii determină o creștere liniară a rezistenței sensorului, ceea ce permite măsurarea exactă a temperaturii.

-Avantaje: Mult mai preciși decât termistoarele NTC sau termocuplurile. RTD-urile nu se degradează rapid, ceea ce le face ideale pentru aplicații unde este necesară măsurarea temperaturii pe perioade lungi.

2.5) Senzorii de temperatură termocuple

Termocuplele sunt un tip de senzor de temperatură care funcționează pe baza efectului thermoelectric. Acest tip de senzor este utilizat pentru a măsura temperaturi foarte ridicate sau în condiții extreme, deoarece poate opera într-un interval de temperatură foarte larg. Această tensiune este proporțională cu diferența de temperatură dintre cele două puncte de conectare.

-Avantaje: Termocuplele oferă un răspuns foarte rapid la schimbările de temperatură datorită dimensiunilor mici ale punctului de contact și a naturii lor fizice. Comparativ cu alți senzori de temperatură, cum ar fi RTD-urile, termocuplele sunt mai ieftine și mai accesibile pentru utilizarea în masă, în special pentru măsurători industriale.

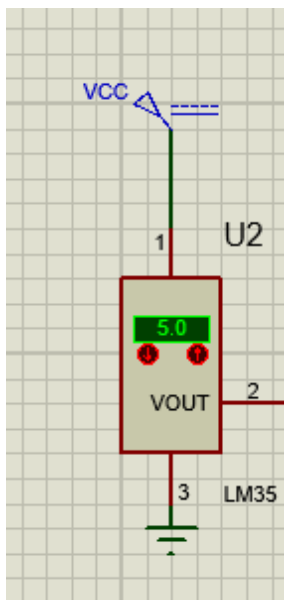
Am ales să folosesc categoria de senzori semiconductori deoarece oferă o liniaritate excelentă și pot fi conectați direct la microcontroler fără circuite suplimentare. Acești senzori asigură o precizie adecvată de 0.1-0.2°C în intervalul de temperatură 5-30°C, sunt ieftini, au un timp de răspuns bun și o durată de viață lungă, ceea ce îi face potriviți pentru aplicații precum termostatele de cameră.

Senzori	Variatia de temperatura	Toleranta	Tensiunea de alimentare	Pret
LM35	-55...150°C	±0.5%	4 ... 30V	2 RON
TMP36	- 60...300°C	±2%	5V	15,22 RON
DS18B20	- 40...180°C	±1%	5V	11,33 RON
LM75	- 30...280°C	1%	5V	19,99 RON
ADT7410	-55...155°C	±1%	5V	35,15 RON

In concluzie, cel mai bun sensor este **LM35** deoarece indeplineste toate cerintele (variatia de temperatura, toleranta mica) si este mai accesibil ca si pret.

3)Senzorul LM35

- Tensiunea variaza direct proportional cu temperatura
- Domeniul temperaturii variaza de la -55°C pana la $+150^{\circ}\text{C}$
- Tensiunea de iesire a senzorului creste liniar cu temperatura, cu un pas de 10mV
- Simbolul senzorului in Proteus:



Stiind ca domeniul dorit este de 5°C pana la 30°C ,calculam tensiunile minime si maxime:

$$\begin{aligned}V_{min} &= (10mV \times 5^{\circ}C) = 50mV \\V_{min} &= (10mV \times 30^{\circ}C) = 300mV\end{aligned}$$

Amplificarea în tensiune este definită ca raportul dintre tensiunea de ieşire şi tensiunea de intrare:

$$A_v = \frac{V_o}{V_{in}} = \frac{5V - 0V}{300mV - 50mV} = \frac{5V}{250mV} = \frac{5V}{0.25V} = 20V$$

Astfel, ştim că domeniul tensiunii de intrare furnizate de senzorul LM35 este între 50 mV (pentru 5°C) şi 300 mV (pentru 30°C), iar domeniul dorit pentru tensiunea de ieşire este 0V - 5V.

Pentru a asigura această conversie, trebuie să determinăm tensiunea de referinţă a amplificatorului operaţional, astfel încât la 5°C ieşirea să fie 0V, iar la 30°C să fie 5V.

Tensiunea de ieşire a AO în configuraţie diferenţială este dată de formula:

$$V_{out} = A_v \cdot (V_{in} - V_{ref})$$

Ştim că la T =5°C, dorim ca Vout = 0V, deci:

$$0V = A_v \cdot (50mV - V_{ref})$$

La T = 30°C, dorim V_out = 5V, deci:

$$5V = A_v \cdot (300mV - V_{ref})$$

4) Dimensionarea rezistentelor:

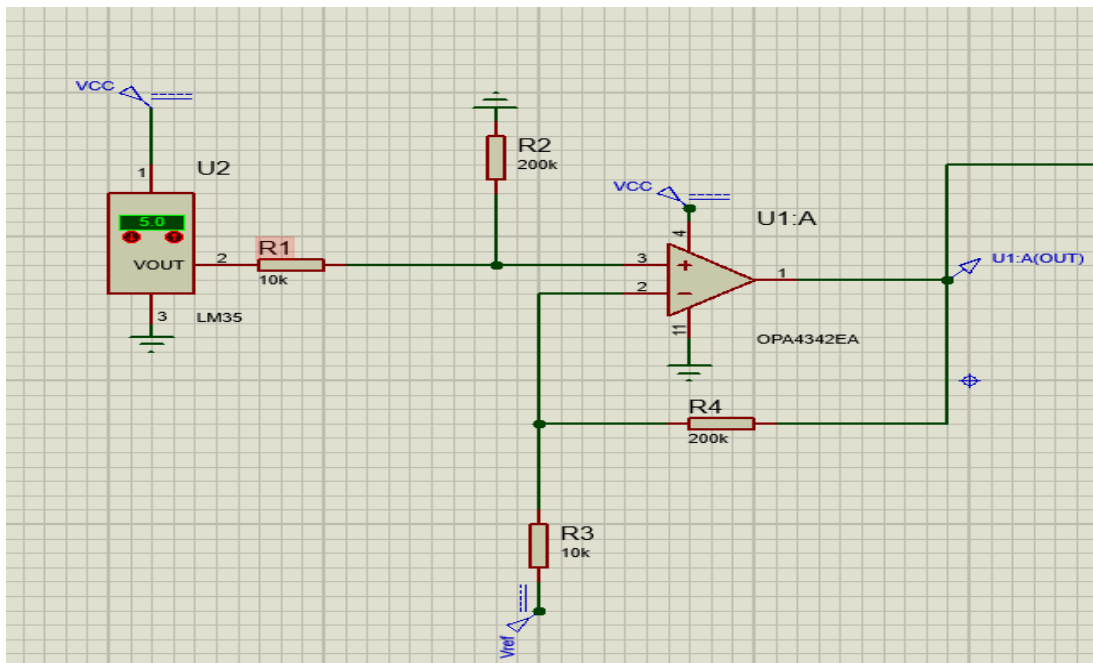
$$\text{Stiind ca: } Av = \frac{R4}{R3} \Rightarrow R4 = Av * R3 = 20 * R3$$

Vom alege valoarea lui $R3 = 10K\Omega$ astfel va rezulta $R4 = 200K\Omega$

Stim ca:

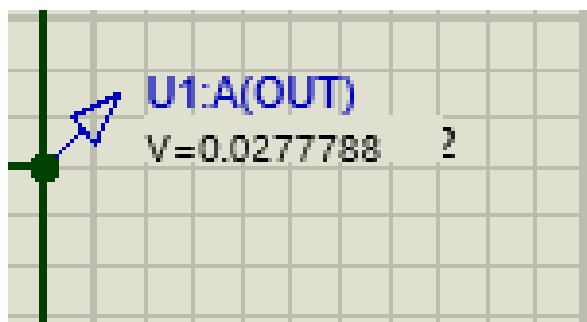
$$R3 = R1 = 10K\Omega \text{ si } R4 = R1 = 200K\Omega$$

-Circuitul cu valorile aflate:

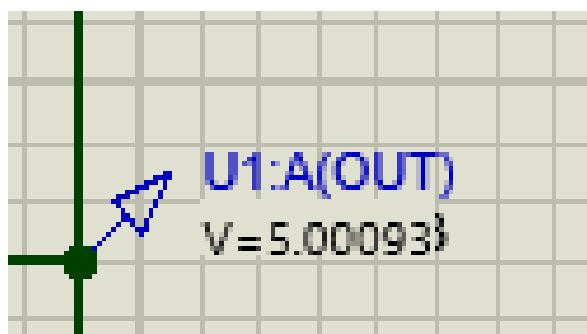


Dupa cum se poate vedea in imaginea urmatoare,atat la 5°C,dar si la 30°C, tensiunile sunt corecte:

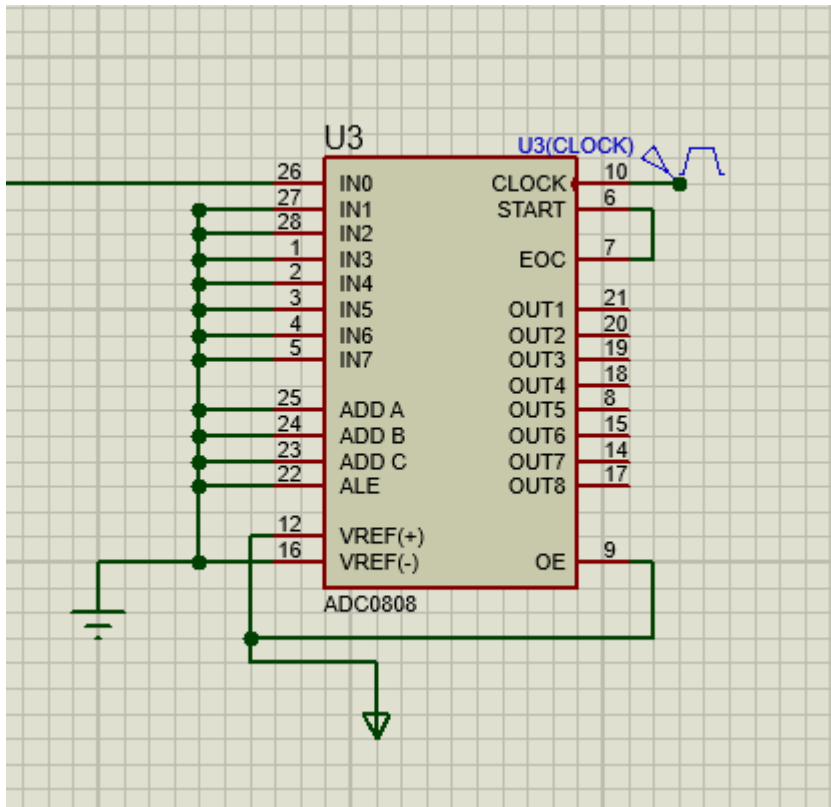
-5°C:



-30°C:



5) Convertorul ADC0808:



Un convertor analog-digital (ADC - Analog-to-Digital Converter) este un dispozitiv electronic care transformă un semnal analogic continuu într-un semnal digital.

În cazul nostru, ADC0808 este un convertor cu 8 biți de rezoluție, ceea ce înseamnă că poate reprezenta semnale analogice prin 256 de valori.

Pentru a determina numărul digital N furnizat la ieșirea convertorului ADC0808, vom folosi următoarea formulă:

$$N = \frac{V}{V_{max}} * N_{max}$$

$V=0V(5^{\circ}C)$ și $5V(30^{\circ}C)$

$V_{max}=5V$

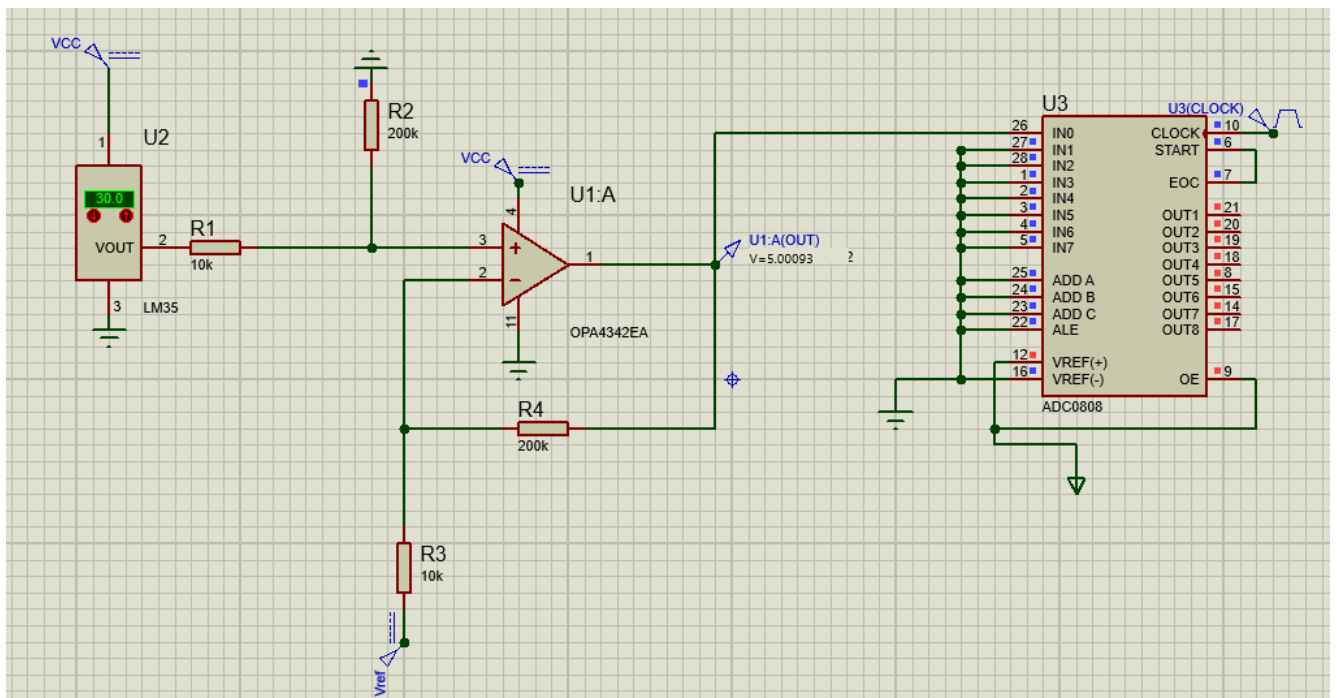
$N_{max}=255$

5

$$N1 = \frac{5}{5} * 255 = FFh$$

$$N2 = - * 255 = 00h$$

-Circuitul complet



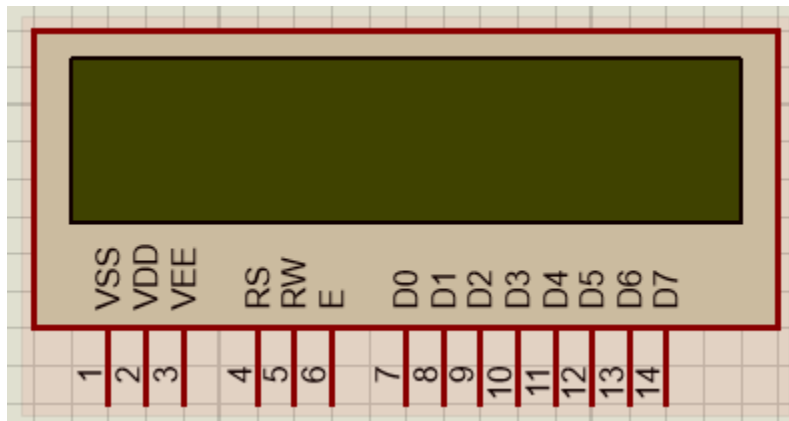
6)Tabel de comparatie pentru alegerea microcontrolerului:

Microcontroler	Memorie(RAM)	Pini I/O	Tensiune alimentare	Pret(RON)
AT80C51RD2-3CSUM	256B	32	2.7 - 5.5V	23.91
AT89LP52-20AU	256B	36	2.7 - 5.5V	8.20
AT89C51-24JI	128B	32	2.7 - 5.5V	19.68
AT89S52	256B	32	2.7 - 5.5V	10.49
STC89C52RC	512B	32	2.7 - 5.5V	5.27

In concluzie,am ales sa utilizez microcontrolerul **STC89C52RC** deoarece are cea mai mare memorie RAM si este si cel mai accesibil ca si pret,iar in aplicatia Proteus am selectat microcontrolerul 80C51.

7)Adaugarea LCD-ului in Proteus:

Am ales sa folosesc in aplicatia Proteus LCD-ul LM016L



LCD-ul (LCD afișaj cu cristal lichid) LM016L este un afișaj LCD foarte popular folosit in special alaturi de microcontrolere precum cele din familia 8051, AVR, PIC sau Arduino.

Functionare:

LCD-ul **LM016L**:

- Este un afișaj alfanumeric cu 16 coloane si 2 linii(16x2).
- Este controlat de un controller care știe să afișeze caractere dintr-un set prestabilit.
- Poți comunica cu el în mod 8-bit (D0-D7) sau mod 4-bit(doar D4-D7).

Componente interne:

- Controller: interpretează comenzile trimise și controlează afișajul.
- Memorie ROM:contine caracterele predefinite(litere de la A la Z,cifre,simboluri)
- Memorie RAM:memorie unde se realizeaza scrierea pentru afisaj

Pentru a testa scrierea in afisajul LCD, am utilizat blocuri logice (3 blocuri logice conectate la RS, RW, E si 8 blocuri logice conectate de la D0 la D7).

Pasii pentru utilizarea si scrierea in LCD:

1. Pentru o pornire corecta, inainte de inceperea simularii, trebuie setata valoarea 0Eh

2. Dupa ce am dat Run, pentru pornirea afisajului, blocul logic conectat la Enable il punem pe pozitia 1, dupa care inapoi pe 0.

3. Pentru a putea scrie, punem in pozitia 1 atat RS cat si E.

4. Folosim cele 8 blocuri logice conectate de la D0 la D7, ajutandu-ne de codul ASCII

R: 0101 0010
I: 0100 1001
M: 0100 1101
B: 0100 0010
U: 0101 0101
L: 0100 1100
E: 0100 0101
A: 0100 0001

R: 0101 0010
O: 0100 1111
B: 0100 0010
E: 0100 0101
R: 0101 0010
T: 0101 0100

G: 0100 0111
R: 0101 0010
U: 0101 0101
P: 0101 0000
A: 0100 0001

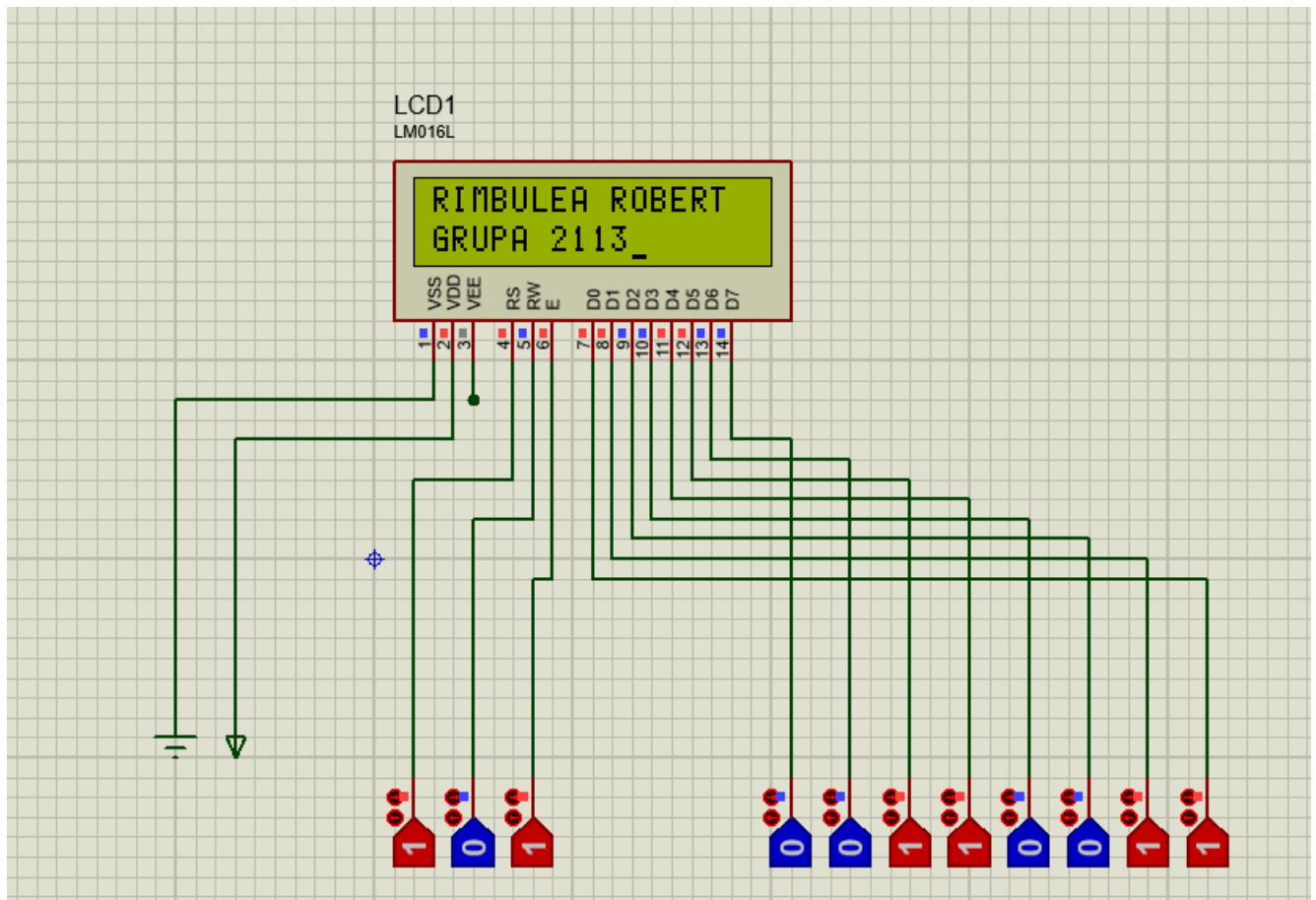
2: 0011 0010
1: 0011 0001
1: 0011 0001
3: 0011 0011

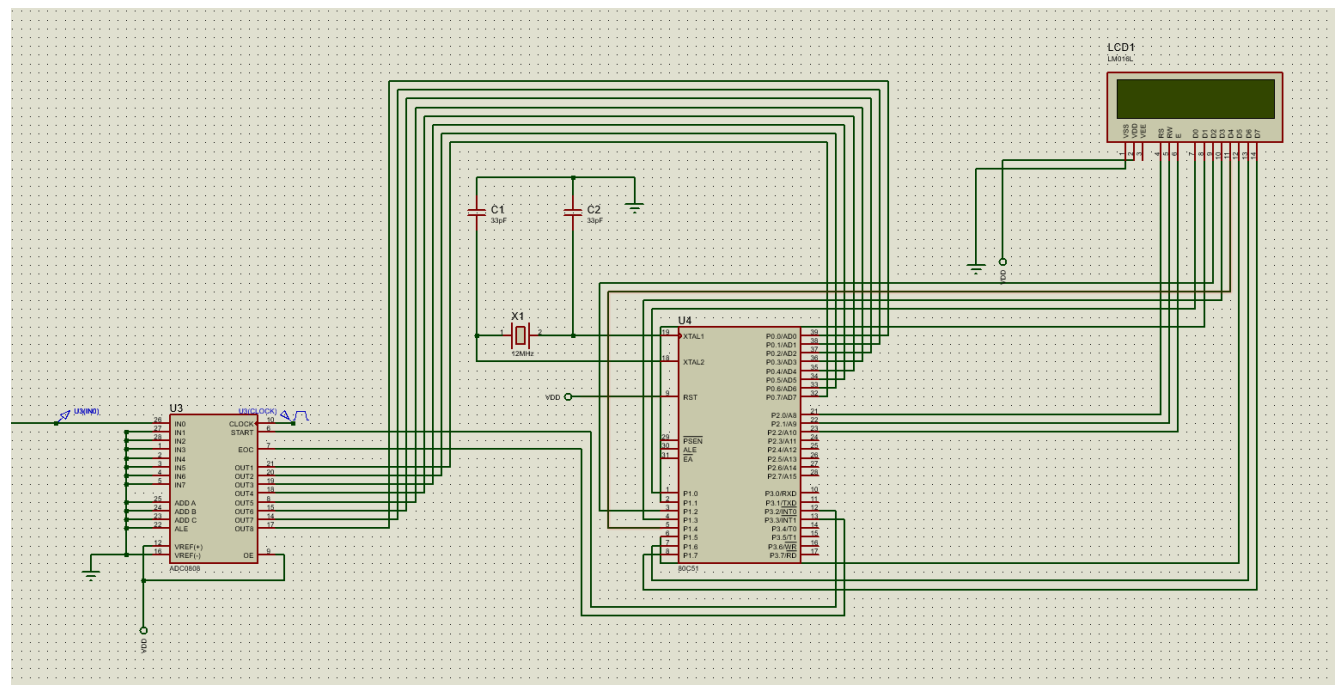
Am mai utilizat urmatoarele comenzi:

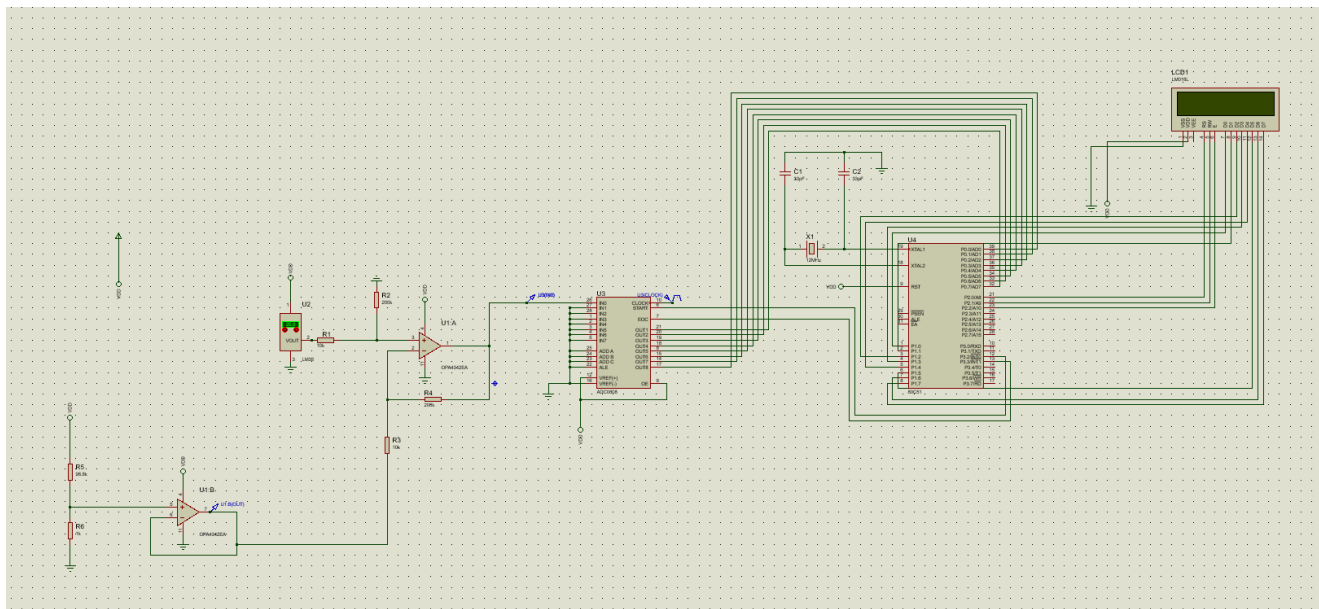
1) 00111000(38h) pentru adaugarea celei de a 2-a linie

- 2) 11000000(C0h) pentru pozitionarea cursorului la dreapta
- 3) 00010100(14h) pentru a muta cursorul pe cea de a 2-a linie

Rezultatul final:





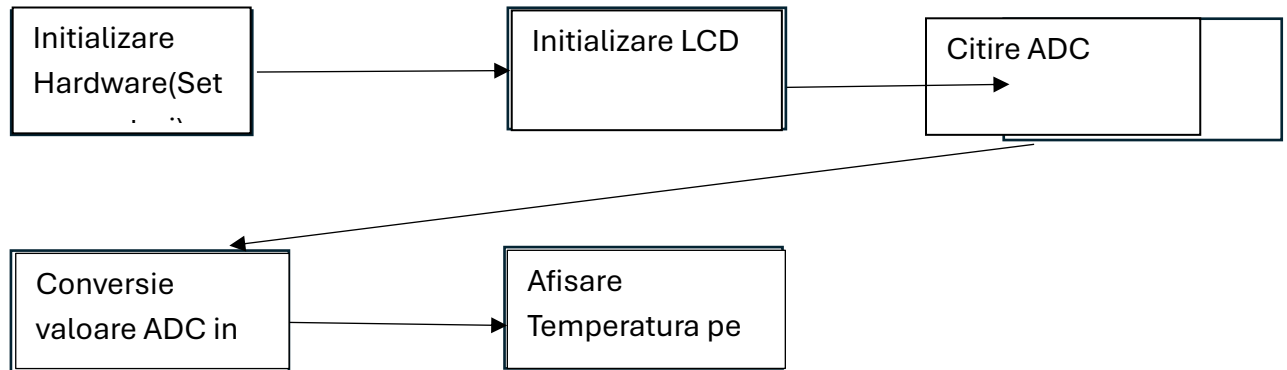


Senzor LM35->CAN ADC0808->Microcontroler 80c51->LCD LM016L

8)Conexiuni

- Senzorul este conectat la ADC in pinul IN0
- Pinii IN1-IN7, ADD A-C, ALE si Vref- sunt conectati la ground
- Vref+ si OE sunt conectati la Vdd
- Iesirile ADC-ului(OUT1-OUT8) sunt conectate la pinii P0.0->P0.7 al microcontrolerului
- Pinul START este conectat la pinul P3.2/INT0 al microcontrolerului
- Pinul EOC este conectat la pinul P3.3/INT1 al microcontrolerului
- Pinii P1.0->P1.7 ai microcontrolerului sunt conectati la pinii D0->D7 al LCD-ului
- Pinul P2.0 al microcontrolerului este conectat la pinul RS al LCD-ului
- Pinul P2.1 al microcontrolerului este conectat la pinul RW al LCD-ului
- Pinul P2.2 al microcontrolerului este conectat la pinul E al LCD-ului
- Am adaugat si 2 condensatoare de 33pf si 1 cristal quartz de 12Mhz pe care le-am conectat la pinii XTAL1 SI XTAL2
- Pinul VSS al LSD-ului este legat la masa-Pinul VDD este legat la VDD

9)Organigrama



10)Codul in C

```
#include <REG51.H>
#define LCD_DATA P1      ( Portul P1 folosit pentru datele LCD)
#define LCD_DATA P1      ( Portul P1 folosit pentru datele LCD)
sbit RS = P2^0;          ( Pinul P2.0 pentru Register Select)
sbit RW = P2^1;          ( Pinul P2.1 pentru Read/Write)
sbit E = P2^2;           (Pinul P2.2 pentru Enable)

#define ADC_DATA P0       (Portul P0 folosit pentru datele ADC)
sbit START = P3^2;        ( Pinul P3.2 pentru a porni conversia ADC)
sbit EOC = P3^3;          (Pinul P3.3 pentru semnalul End Of Conversion)
```

unsigned char temp = 0; (Variabilă pentru stocarea temperaturii)

(Funcție pentru întârziere în milisecunde)

```
void delay_ms(unsigned int ms) {
    unsigned int i, j;
    for(i = 0; i < ms; i++)      (Buclă pentru fiecare milisecundă)
        for(j = 0; j < 1275; j++); (Buclă internă pentru întârziere precisă)
}
```

(Funcție pentru trimiterea unei comenzi la LCD)

```
void lcd_command(unsigned char cmd) {
    RS = 0;      (Setează modul comandă (RS=0))
```

```

    RW = 0;          (Setează modul scriere (RW=0))
    LCD_DATA = cmd;   (Trimite comanda pe portul de date)
    E = 1; delay_ms(1); (Activează semnalul Enable cu întârziere)
    E = 0; delay_ms(1); (Dezactivează semnalul Enable)
}

```

(Funcție pentru trimiterea datelor la LCD)

```

void lcd_data(unsigned char dat) {
    RS = 1;          (Setează modul date (RS=1))
    RW = 0;          (Setează modul scriere (RW=0))
    LCD_DATA = dat;   (Trimite datele pe portul de date)
    E = 1; delay_ms(1); (Activează semnalul Enable cu întârziere)
    E = 0; delay_ms(1); (Dezactivează semnalul Enable)
}

```

(Funcție de inițializare LCD)

```

void lcd_init() {
    delay_ms(50);      (Așteaptă stabilizarea tensiunii LCD)
    lcd_command(0x38);  (Setează modul 8-bit, 2 linii, font 5x8)
    lcd_command(0x0C);  (Pornire display, cursor off, blink off)
    lcd_command(0x01);  (Șterge display)
    delay_ms(5);
    lcd_command(0x06);  (Incrementare cursor, fără shift display)
}

```

(Funcție pentru afișarea temperaturii pe LCD)

```

void lcd_display_temp(unsigned char t) {
    lcd_command(0x80);  (Poziționează cursorul la începutul primei linii)
    lcd_data('T'); lcd_data('e'); lcd_data('m'); lcd_data('p'); lcd_data(':'); (Scrie "Temp")
    lcd_data((t / 10) + '0'); (Afișează cifra zecilor)
    lcd_data((t % 10) + '0'); (Afișează cifra
unităților)
    lcd_data(0xDF);      (Simbolul grade (°))
    lcd_data('C');       (Litera C pentru
Celsius)
}

```

(Funcție pentru citirea valorii de la ADC)

```

unsigned char read_adc() {
    START = 1;          (Pornește conversia ADC)
    delay_ms(1);        (Întârziere scurtă)
    START = 0;          (Oprește semnalul de start)
}

```

```

while(EOC == 0);    (Așteaptă finalizarea conversiei (EOC=1))
delay_ms(1);        (Întârziere scurtă)

return ADC_DATA;    (Returnează valoarea convertită)
}

( Funcția principală)

void main() {
    unsigned char raw;    (Variabilă pentru valoarea brută de la ADC)

    ADC_DATA = 0xFF;    (Configurează P0 ca port de intrare)
    lcd_init();          (Inițializează LCD-ul)

    while(1) {           ( Buclă infinită)
        raw = read_adc();    (Citește valoarea de la ADC)

        ( Conversie din valoare ADC în temperatură (5° C - 30° C))
        if(raw <= 2)
            temp = 5;        (Valoare minimă (5° C))
        else if (raw >= 253)
            temp = 30;        (Valoare maximă (30° C))
        else

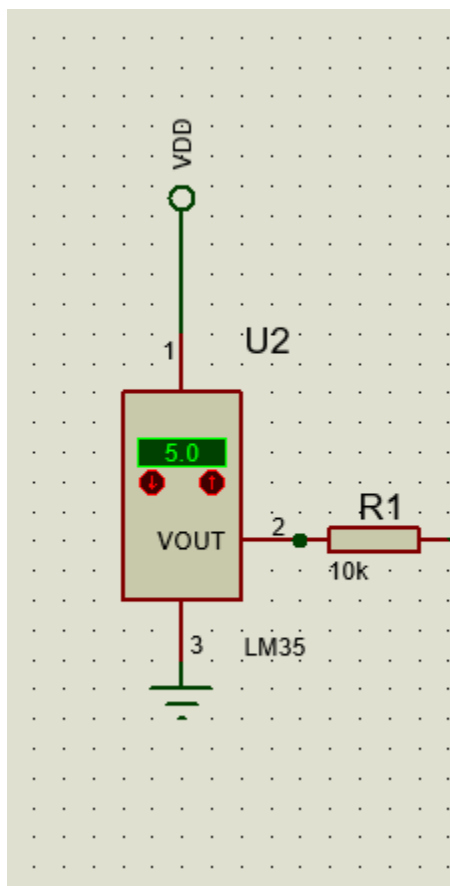
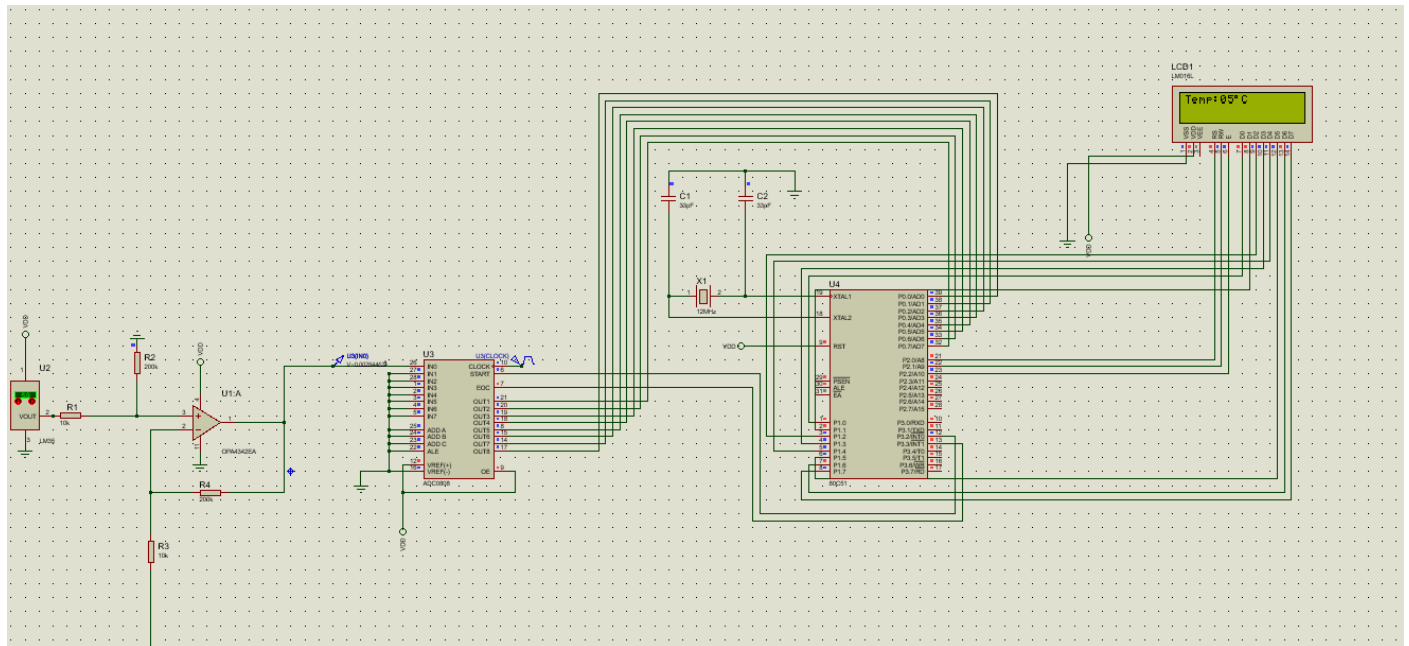
            ( Interpolare liniară între 5° C și 30° C)

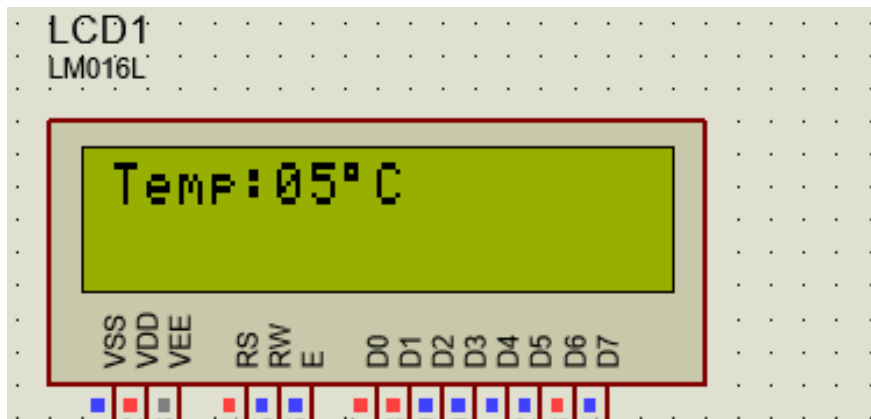
            temp = 5 + ((raw - 2) * 25) / 251;

        lcd_display_temp(temp);    (Afișează temperatura pe LCD)
        delay_ms(500);            (Întârziere între citiri)
    }
}

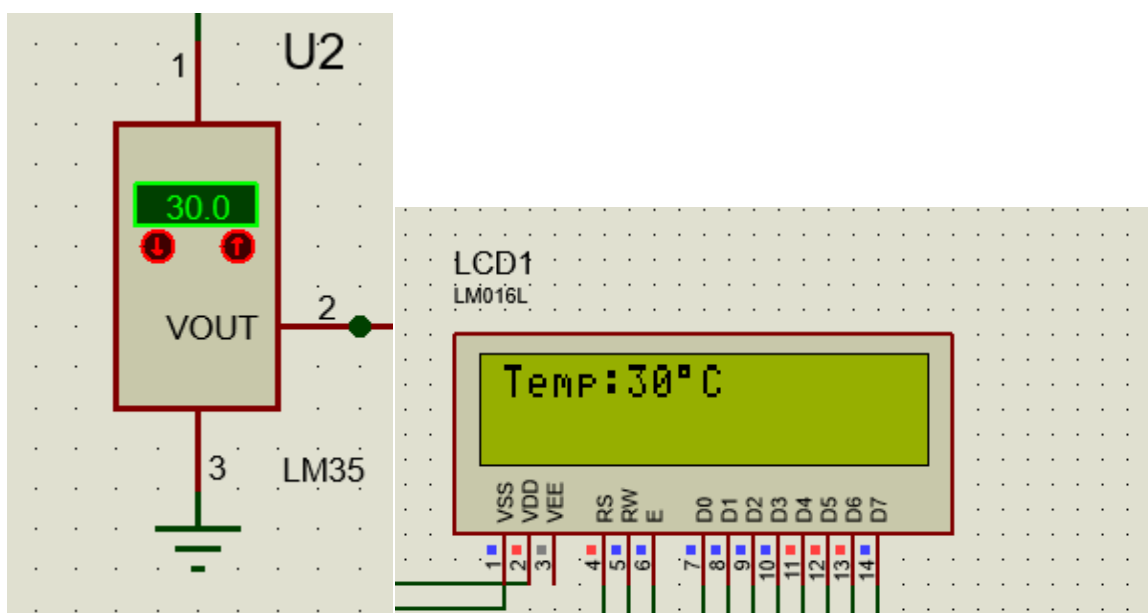
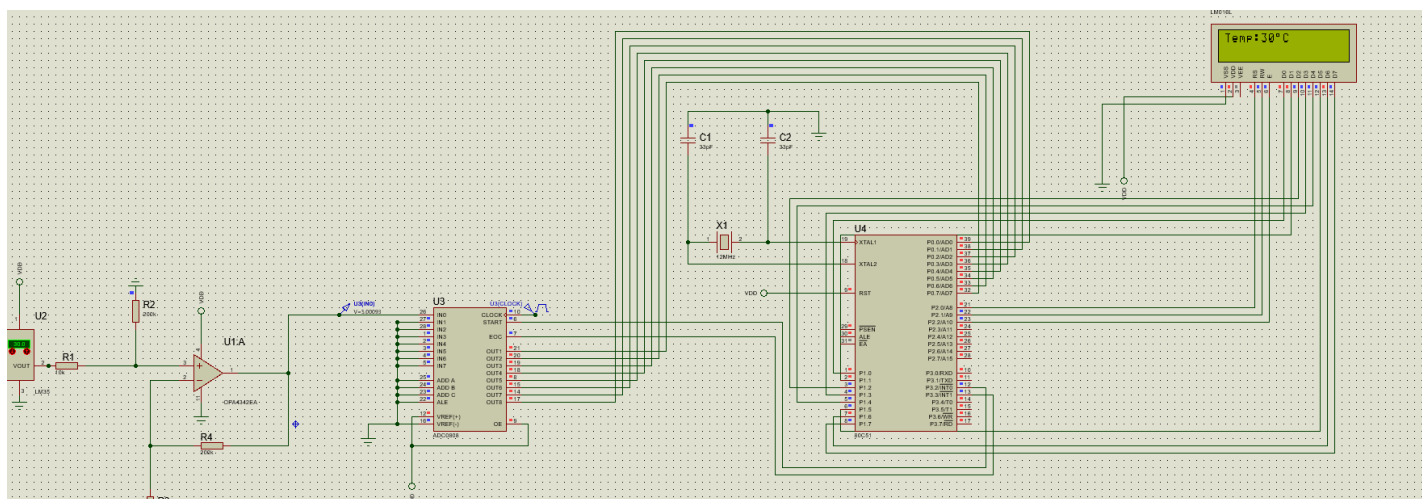
```

11) Simularea la 5 grade

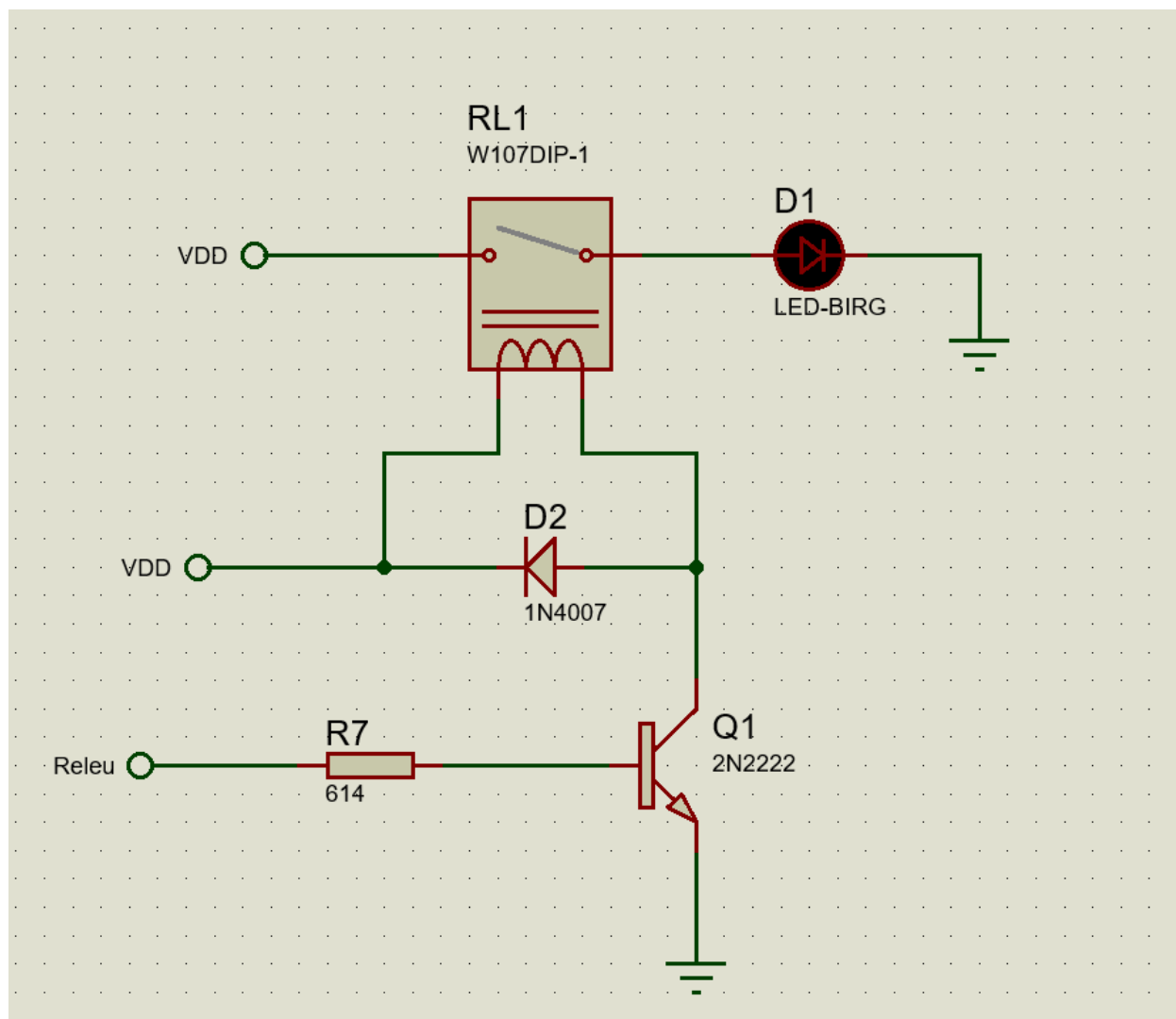




12) Simularea la 30 grade



13) Adaugarea Releului in Proteus



Am ales sa folosesc releul W107DIP-1 deoarece acesta oferă un comutator cu contacte uscate (dry reed) care asigură o durabilitate și fiabilitate ridicată în aplicații de comutare cu curenți mici, iar bobina de 5V este compatibilă cu nivelurile de tensiune ale microcontrolerului utilizat, facilitând astfel integrarea simplă și eficientă în circuit.

Am folosit tranzistorul 2N2222 deoarece acesta funcționează ca un comutator eficient ce permite microcontrolerului 80C51 să controleze releul.

Am ales sa folosesc dioda 1N4007 deoarece diodă redresoare robustă, suficientă pentru majoritatea aplicațiilor cu releu de curent mic/mediu.

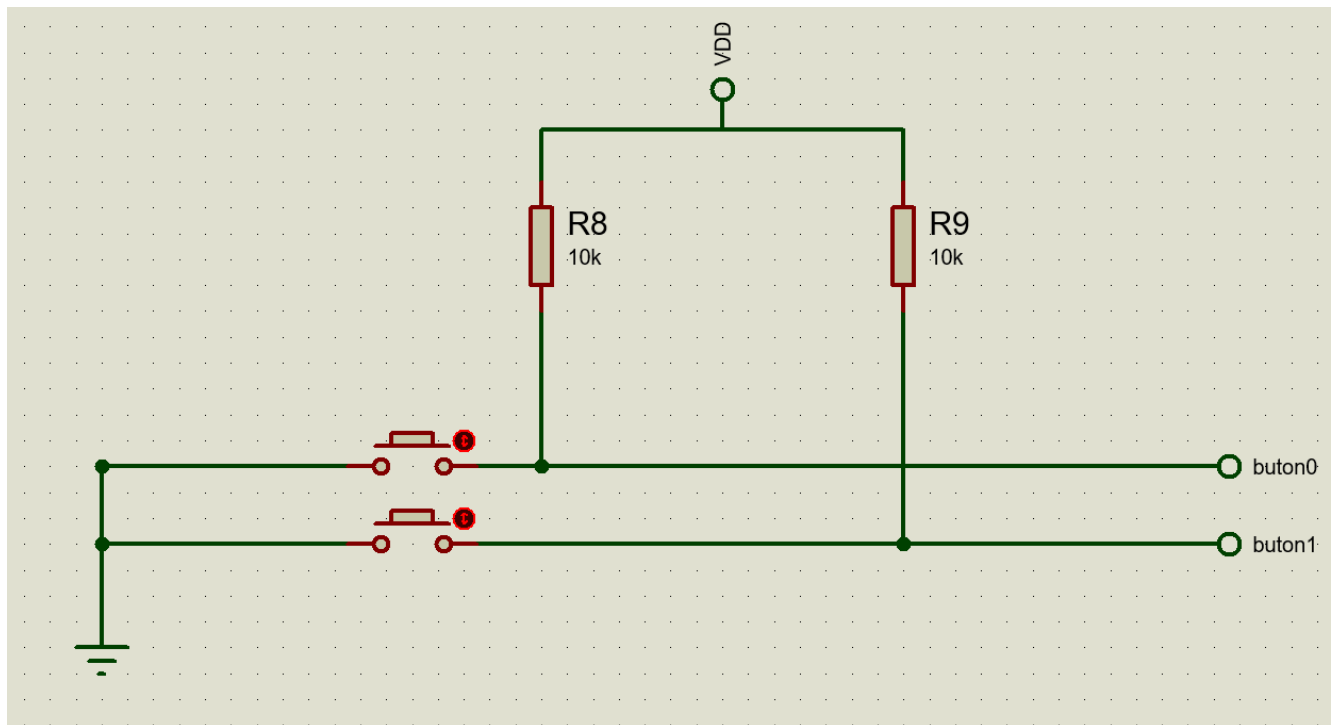
-Aflarea rezistenței R7

Având VCC de 5V și folosind tensiunea bază-emitor a tranzistorului de 0.7V, am calculat tensiunea pe rezistență : $5V - 0.7V = 3.4V$

Am estimat curentul de colector la 70 mA pentru că este o valoare tipică pentru relele folosite frecvent în circuite electronice.

Folosind legea lui Ohm, am calculat rezistența: $R = \frac{4.3V}{7mA} = 614\Omega$.

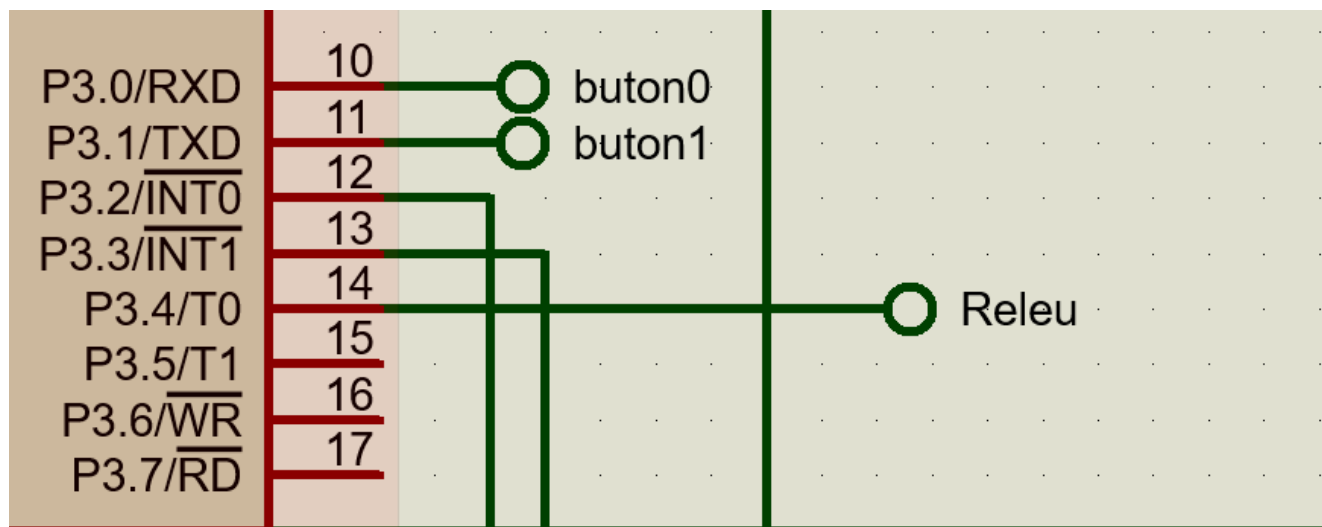
14)Adaugarea Butoanelor in Proteus



Aici avem 2 rezistente de 10kΩ conectate între sursa de tensiune VDD și intrările la care sunt apasate butoanele.

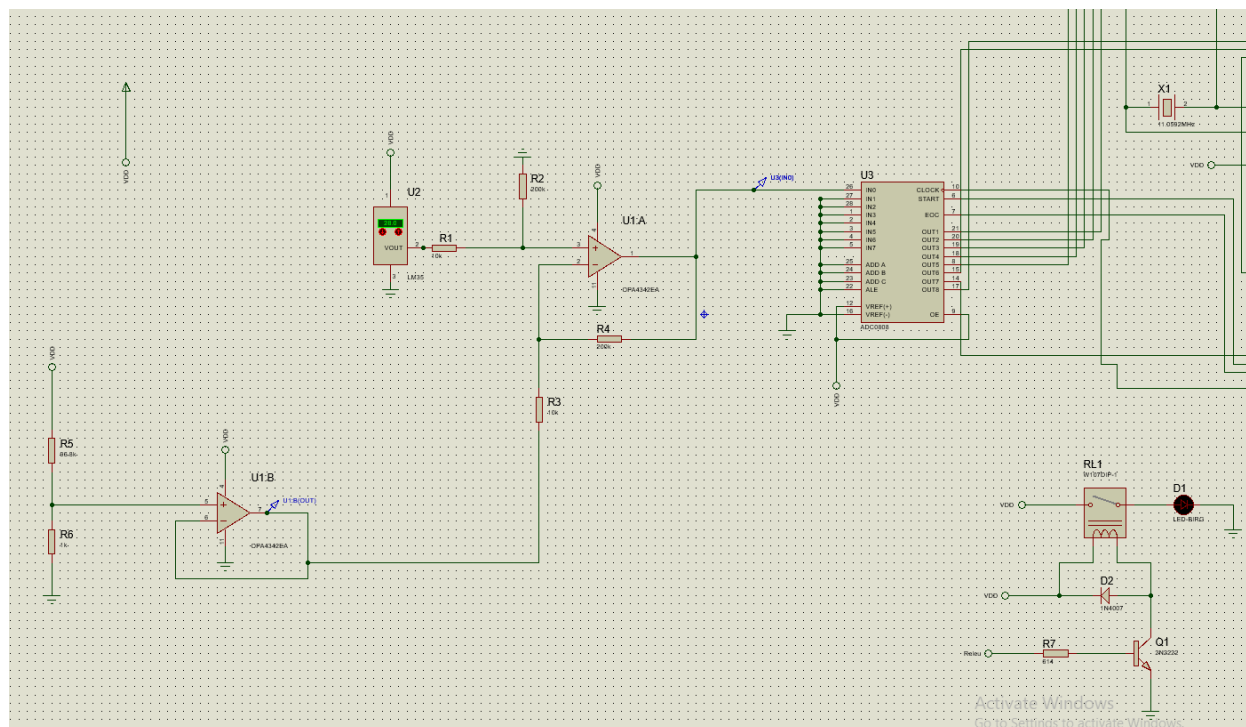
Butonul 0 este folosit pentru a crește temperatura pe LCD

Butonul 1 este folosit pentru a scăde temperatura pe LCD



Butonul 1 l-am conectat la pinul P3.0 al microcontrolerului
 Butonul 2 l-am conectat la pinul P3.1 al microcontrolerului
 Releul l-am conectat la pinul P3.4 al microcontrolerului

15)Circuitul complet




```

21 // === Semnal de ceas pentru ADC ===
22 sbit CLOCK = P3^5;           // CLOCK - semnal de ceas pentru ADC generat de Timer0
23
24 // === Variabile globale ===
25 unsigned char temp = 0;       // Temperatura actuala citita
26 unsigned char temp_set = 25;  // Temperatura setata inițial (25°C)
27
28 // === Funcție pentru întârziere în milisecunde ===
29 void delay_ms(unsigned int ms) {
30     unsigned int i, j;
31     for(i = 0; i < ms; i++)
32         for(j = 0; j < 1275; j++); // Delay aproximativ de 1 ms
33 }
34
35 // === Trimite o comanda catre LCD ===
36 void lcd_command(unsigned char cmd) {
37     RS = 0;                     // Selecteaza registrul de comanda
38     RW = 0;                     // Selecteaza scrierea
39     LCD_DATA = cmd;             // Trimite comanda pe port
40     E = 1; delay_ms(1);         // Pulseaza Enable
41     E = 0; delay_ms(1);
42 }
43
44 // === Trimite un caracter catre LCD ===
45 void lcd_data(unsigned char dat) {
46     RS = 1;                     // Selecteaza registrul de date
47     RW = 0;                     // Selecteaza scrierea
48     LCD_DATA = dat;             // Trimite caracterul
49     E = 1; delay_ms(1);         // Pulseaza Enable
50     E = 0; delay_ms(1);
51 }
52
53 // === Inițializeaza LCD-ul în mod 8-bit, 2 linii ===
54 void lcd_init() {
55     delay_ms(50);               // Așteapta ca LCD-ul sa se stabilizeze
56     lcd_command(0x38);          // Mod 8-bit, 2 linii, 5x7 caractere
57     lcd_command(0x0C);          // Afișaj ON, cursor OFF
58     lcd_command(0x01);          // Șterge ecranul
59     delay_ms(5);                // Timp pentru ștergere
60     lcd_command(0x06);          // Cursor la dreapta dupa fiecare caracter
61 }
62
63 // === Afișează temperatura actuală și temperatura setată pe LCD ===
64 void lcd_display_temp(unsigned char t) {
65     lcd_command(0x80);          // Cursor pe prima linie
66     lcd_data('T'); lcd_data('e'); lcd_data('m'); lcd_data('p'); lcd_data(':');
67     lcd_data((t / 10) + '0');   // Zeci

```

```

67     lcd_data((t / 10) + '0');           // Zeci
68     lcd_data((t % 10) + '0');           // Unitate
69     lcd_data(0xDF);                     // Simbolul ° (grad)
70     lcd_data('C');
71
72     lcd_command(0xC0);                   // Cursor pe a doua linie
73     lcd_data('S'); lcd_data('e'); lcd_data('t'); lcd_data(':');
74     lcd_data((temp_set / 10) + '0');     // Zeci
75     lcd_data((temp_set % 10) + '0');     // Unitate
76     lcd_data(0xDF);                     // Simbolul °
77     lcd_data('C');
78 }
79
80 // === Citește valoarea de la ADC ===
81 unsigned char read_adc() {
82     START = 1;                           // Pornește conversia ADC
83     delay_ms(1);
84     START = 0;
85
86     while(EOC == 0);                     // Așteapta terminarea conversiei
87     delay_ms(1);
88
89     return ADC_DATA;                     // Returnează valoarea convertită
90 }
91
92 // === Rutina de întrerupere pentru Timer0 (pentru semnalul de clock al ADC) ===
93 void Timer0(void) interrupt 1 {
94     CLOCK = !CLOCK;                       // Inversează semnalul de clock (toggle)
95     TH0 = 0xFF;                           // Reîncarcă Timer0 pentru ~5 μs
96     TL0 = 0xFB;
97 }
98
99 void main() {
100     unsigned char raw;                   // Valoarea brută de la ADC
101     bit prev_btn_inc = 1;                // Starea anterioară a butonului de creștere
102     bit prev_btn_dec = 1;                // Starea anterioară a butonului de scădere
103
104     ADC_DATA = 0xFF;                     // Setează portul P0 ca intrare (tristate)
105     RELAY = 0;                           // Opriți releul la început
106     P3 |= 0x03;                           // Activează rezistențe de pull-up pentru P3.0 și P3.1
107
108     lcd_init();                           // Inițializează LCD-ul
109
110     // === Inițializare Timer0 pentru generare semnal CLOCK ===
111     TMOD &= 0xF0;                         // Curăță setările Timer0
112     TMOD |= 0x01;                         // Setează Timer0 în mod 1 (16-bit)
113     TH0 = 0xFF;                           // Setează Timer0 pentru perioada scurtă
114     TL0 = 0xFB;
115     ET0 = 1;                             // Activează întreruperea Timer0
116     EA = 1;                             // Activează întreruperile globale
117     TR0 = 1;                             // Pornește Timer0
118
119     while(1) {
120         raw = read_adc();                 // Citește valoarea brută de la ADC
121
122         // Conversie brută -> temperatură (mapare între 5°C și 30°C)
123         if(raw <= 2)
124             temp = 5;
125         else if (raw >= 253)
126             temp = 30;
127         else
128             temp = 5 + ((raw - 2) * 25) / 251;
129     }
130 }

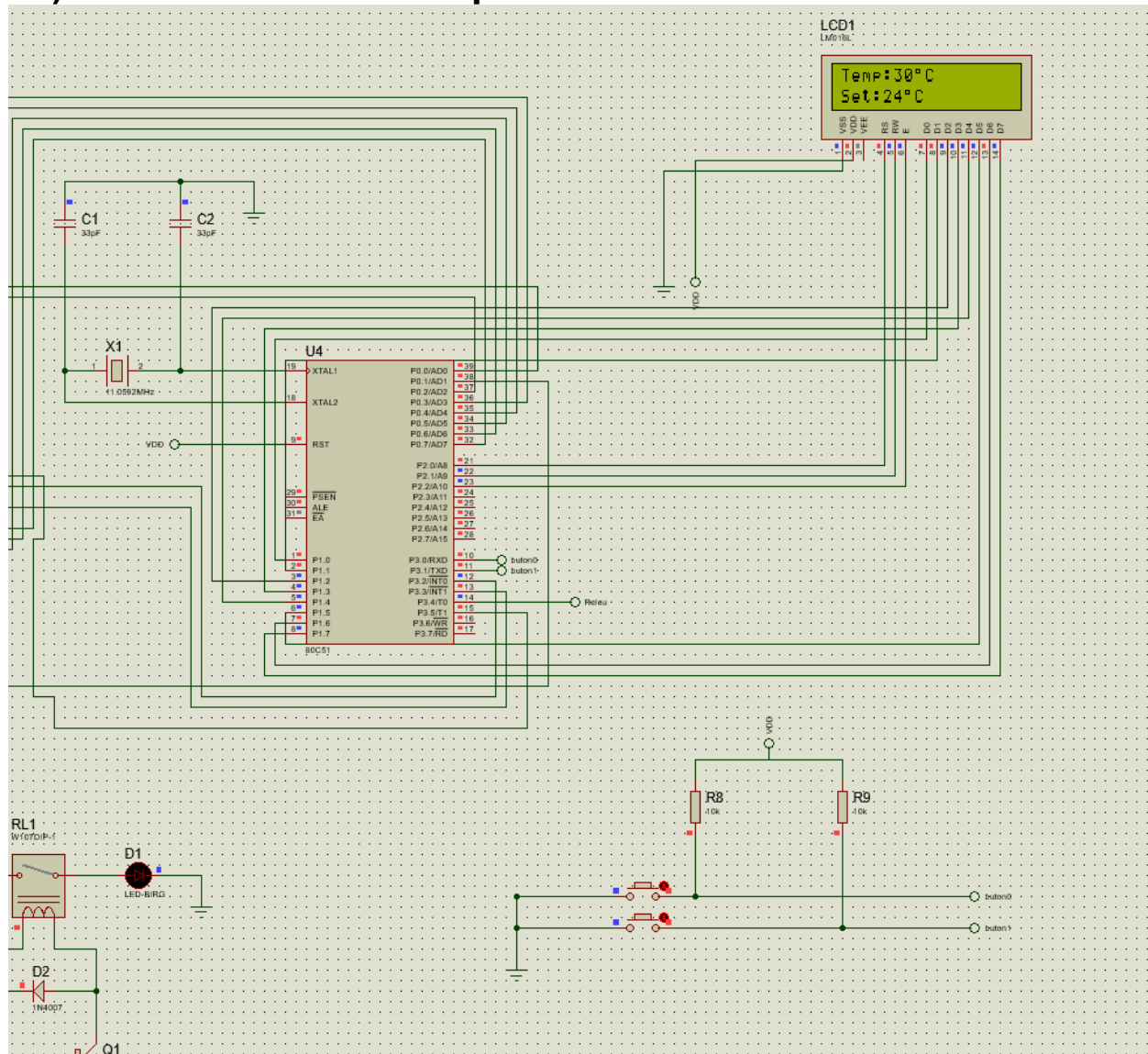
```

```

131 // Detecteaza o noua apasare a butonului de creștere (tranziție 1->0)
132 if (BTN_INC == 0 && prev_btn_inc == 1) {
133     if (temp_set < 99)
134         temp_set++; // Crește temperatura setata
135 }
136 prev_btn_inc = BTN_INC;
137
138 // Detecteaza o noua apasare a butonului de scadere (tranziție 1->0)
139 if (BTN_DEC == 0 && prev_btn_dec == 1) {
140     if (temp_set > 0)
141         temp_set--; // Scade temperatura setata
142 }
143 prev_btn_dec = BTN_DEC;
144
145 // Controlul releului cu histerezis de ±1°C
146 if(temp < temp_set - 1)
147     RELAY = 1; // Pornește releul (încalzire)
148 else if(temp > temp_set + 1)
149     RELAY = 0; // Oprește releul
150
151 lcd_display_temp(temp); // Afișează temperaturile pe LCD
152 delay_ms(100); // Întârziere pentru stabilitate vizuala
153 }
154 }
155

```

17) Verificarea codului in proteus



Cand temperatura setata este mai mica decat temperatura aleasa din senzorul LM35, se poate observa ca ledul de la releu este stans

